LFX MENTORSHIP PROPOSAL (2026 TERM 3)

CNCF Meshery: MCP Server | Issue #19446

Applicant: Peaush Paul	**Email: peaushpaul99@gmail.com**
Location: Kolkata, India (UTC+5:30)	**CNCF Slack: @PeaushPaul**
GitHub: github.com/peaush07	**LinkedIn:** linkedin.com/in/peaush07
Live PoC Repository: github.com/peaush07/meshery-mcp-server-poc	**Mentorship Term: 2026 Term 3 (Sep – Nov)**

1. Executive Summary

As AI-assisted software development tools (Claude Desktop, Cursor IDE, VS Code AI agents) become standard across cloud engineering, developers face a critical context gap when interacting with Meshery: AI agents currently have no native protocol connection to list infrastructure designs, inspect live Kubernetes clusters, query model registries, or execute performance tests. Developers must manually copy YAML designs back and forth.

This proposal presents the architecture, implementation plan, and verified **Golang Proof-of-Concept** for the official **Meshery Model Context Protocol (MCP) Server** (meshery-mcp-server). Built on Meshery schemas, **MeshKit**, and MeshSync topology, this project exposes Tools, Read-Only Resources, and Guided Prompts over **stdio** and **streamable HTTP (SSE)** transports under hard security constraints (zero state mutations and zero secret leakage), targeting >=80% test coverage and multi-platform release.

2. Technical Architecture & MCP Execution Flow

```
[ AI CLIENT (Cursor / Claude) ] --( JSON-RPC 2.0 stdio/SSE )--> [ MESHERY MCP SERVER (Go) ]
|
+------------------------------------------------------------------+------------------------------------------
--------+
| [MCP TOOLS: list_designs, run_test] [MCP RESOURCES: meshsync://topology] [SECURITY: Secret Redactor Engine]
|
+------------------------------------------------------------------+------------------------------------------
--------+
|
( Meshery REST/GraphQL via MeshKit )
v
[ MESHERY CONTROL PLANE (K8s/MeshSync) ]
```

Execution Component	How MCP Runs Under the Hood
1. Handshake & Capabilities	On boot, AI client connects via stdio pipe. Server responds to initialize request with JSON-RPC capabilities listing supported tools, resources, and prompts.
2. Tool Execution (`tools/call`)	When user requests infrastructure state, AI sends JSON-RPC tool call. Server validates schemas, calls Meshery REST API (`/api/content/patterns`), and returns structured JSON.
3. Resource Streaming (`meshsync://`)	Streams read-only MeshSync cluster topology. Intercepted by `pkg/security` sanitizer to automatically redact tokens, passwords, and K8s secret payloads.
4. Guided Workflow Prompts	Provides prompt templates (`investigate_cluster_health`, `review_design`) to guide AI agents through complex infrastructure analysis without open-ended mutating risks.

3. 12-Week Implementation Plan & Mentor Review Milestones

Phase	Timeline	Key Milestones & Focus	Deliverables & Demos
Phase 1	Weeks 1–2	Repo scaffolding, MeshKit setup, stdio transport baseline, config manager.	Working CLI binary scaffold.
Phase 2	Weeks 3–5	Meshery Go REST/GraphQL client, core MCP Tools (Designs, Clusters, Registry, Nighthawk).	MCP Tools suite + schemas.
Phase 3	Weeks 6–7	Read-only MeshSync topology & Relationship resources, secret masking sanitizer package.	Sanitized MeshSync resources.
Phase 4	Weeks 8–9	MCP Prompts implementation, verification against Claude Desktop and Cursor IDE.	Prompts & client configs.
Phase 5	Weeks 10–11	Achieve >=80% test coverage, setup GitHub Actions CI/CD with GoReleaser & Docker multi-arch.	CI/CD release workflows.
Phase 6	Week 12	Final user quickstart guide (<10 min setup), publishing to MCP/Skills marketplaces, live demo.	Published docs & video demo.

4. Why Choose Me? (Live PoC Demo & Value Proposition)

1. Verified Working Proof of Concept (Published Demo)

Prior to selection, I architected and published a functional **Golang PoC** repository (**github.com/peaush07/meshery-mcp-server-poc**) with stdio/SSE transports and secret sanitization.

2. Deep Technical Alignment

Hands-on expertise in Golang microservices, Kubernetes APIs, REST/GraphQL clients, and Anthropic MCP specifications.

3. Long-Term Maintainership Commitment

Dedicated to becoming an official long-term maintainer of meshery-mcp-server post-mentorship as requested in Issue #19446.

4. Self-Directed Quality

Focus on >=80% test coverage, multi-arch CI/CD automation, and active participation in weekly Meshery community meetings.

5. Relevant Experience & Live PoC Repository

- **Golang Systems Engineering:** Building concurrent Go apps, CLI binaries, HTTP/SSE servers, and API wrappers.
- **Cloud-Native Infrastructure:** Kubernetes API, MeshSync topology, Docker containerization, and CNCF standards.
- **Live PoC Demo Repo: https://github.com/peaush07/meshery-mcp-server-poc**

6. Mentorship Commitment & Verification

If selected for CNCF Meshery MCP Server mentorship (2026 Term 3), I dedicate **40+ hours per week** to achieving all milestones, attending weekly community meetings, incorporating maintainer feedback, and maintaining the project long-term as an official maintainer.

Applicant Signature: Peaush Paul
Kolkata, West Bengal, India | peaushpaul99@gmail.com

Target Program: CNCF Meshery — LFX Mentorship 2026 Term 3
Issue #19446 | meshery-mcp-server